Enable variable template template parameters

Document #:	P2008R0
Date:	2020-01-10
Project:	Programming Language C++ Evolution Working Group Incubator

Reply-to:	Mateusz Pusz (Epam Systems) <mateusz.pusz@gmail.com>

Contents

§ 1 Introduction

In C++ we can easily form a template that will take a class template as its parameter. With C++14 we got variable templates. They proved to be useful in many domains but still are not first-class citizens in C++. For example, they cannot be passed as a template parameter to a template. This document proposes adding such a feature.

§ 2 Motivation and Scope

To check if the current type is an instantiation of a class template we can write the following type trait:

```
template<typename T>
struct is_ratio : std::false_type {};

template<intmax_t Num, intmax_t Den>
struct is_ratio<std::ratio<Num, Den>> : std::true_type {};
```

A family of such type traits can be passed to a concept:

```
template<typename T, template<typename> typename Trait>
concept satisfies = Trait<T>::value;
```

and then be used to constrain a template:

```
template<satisfies<is_ratio> R1, satisfies<is_ratio> R2>
using ratio_add = /* ... */;
```

After C++14 we learnt that variable templates are less to type (no need for a `_v` helper) and are often faster to compile. The above `is_ratio` type trait can be rewritten as:

```
template<typename T>
inline constexpr bool is_ratio = false;

template<intmax_t Num, intmax_t Den>
inline constexpr bool is_ratio<ratio<Num, Den>> = true;
```

However, contrary to a class template, the above variable template cannot be passed to a template at all. The syntax:

```
template<typename T, template<typename> bool Trait>
concept satisfies = Trait<T>;
```

is not a valid C++ as of today.

§ 3 Proposal

Extend C++ template syntax with a variable template parameter kind.

§ 4 Acknowledgements

Special thanks and recognition goes to Epam Systems for supporting my membership in the ISO C++ Committee and the production of this proposal.